SINGLE-PROJECT FULL-STACK + AI ROADMAP

Ascent

27 weeks. Not 27 different toy projects—but one single, real, growing AI-powered SaaS project where each week's new learning is directly integrated into the same application. From Week 1's closures to Week 27's RAG, everything is within the same codebase and follows the same story.

- **PHASE 1:** Core Foundations (Week 1-6)
- **PHASE 2:** Frontend Engineering (Week 7-14)
- **PHASE 3:** Backend Engineering (Week 15-22)
- **PHASE 4:** AI Integration (Week 23-27)

Prepared for: A CSE graduate rebuilding their career—restarting from Week 1, with intent.

Date: June 2026

Why Only One Project?

- Building a new, isolated project every week keeps the learning superficial.
- The concept gets tested, but understanding *why it is needed* is missed because the project is discarded the following week.
- Instead, in this guide, a single app—**Ascent**—will be built continuously over the course of 27 weeks.
- Week 1's closure-based task logic transforms into React state in Week 8; Week 17's database becomes the raw material for AI in Week 24.
- Each week builds upon the previous one—so by Week 27, you will have a deep, interview-ready, real-world product in your hands. Not 27 scattered repositories.

Ascent — Project Concept

Ascent is an AI-powered personal learning and productivity platform—a SaaS where users can track their roadmaps, keep notes, upload study materials to chat with an AI Mentor, and share their learning space with others in real-time. (You can change the name later; the concept is what matters) .

- **Roadmap / Task Tracker:** Full CRUD with priority, due date, and status.

- **Learning Journal:** MDX blog for posting weekly learnings.
- **Document Upload:** Uploading study materials (PDFs/notes).
- **AI Mentor:** Contextual answers with citations derived from uploaded documents (RAG).
- **Smart Summarizer:** Summarizing long notes/documents.
- **Study Groups:** Real-time collaboration with others.
- **Auth:** Private, secure data dedicated to each individual user.
- **Admin Dashboard:** Usage statistics and subscription management.

Architecture — How the 4 Phases Fit Together

1. **AI Layer — RAG, Streaming, Tool Calling** (Week 23-27)
    - *Sits on top of:*
2. **Backend — Express, PostgreSQL, Redis, Auth** (Week 15-22)
    - *Sits on top of:*
3. **Frontend — React, Next.js, Tailwind** (Week 7-14)
    - *Sits on top of:*
4. **Core Engine — JS/TS Fundamentals, Types, Repo Skeleton** (Week 1-6)

Tech Stack Summary

Layer	Technology	When It Appears
Language	JavaScript (ES2024), TypeScript	Week 1-6
Frontend	React, Next.js (App Router)	Week 7-14
Styling	Tailwind CSS, CSS Modules	Week 7, 12
State Mgmt	Zustand, TanStack Query	Week 11

Layer	Technology	When It Appears
Backend	Node.js, Express.js	Week 15-16
Database	PostgreSQL, Prisma ORM	Week 17
Auth/Security	JWT, bcrypt	Week 18
Caching/Queue	Redis, BullMQ	Week 19
Realtime	Socket.io (WebSocket)	Week 19, 22
AI	OpenAI API, Vercel AI SDK, pgvector	Week 23-25
DevOps	Docker, GitHub Actions, Vercel/Railway	Week 21
Testing	Vitest, React Testing Library, Playwright	Week 13

PHASE 1: Core Foundations (Week 1-6)

In this phase, Ascent has no UI or server. Only the core logic modules and data models will be built, which will be reused repeatedly over the next 21 weeks.

Week 1: JavaScript Engine — Scope & Closures

- **What you will build in ASCENT:** The core logic of Ascent's Task Tracker. A closure-based module named createTaskStore() that stores tasks in-memory and keeps the state of each task private (no class, no external library). Along with it, 5 small demos to understand closures (counter factory, debounce function, module pattern).
- **Core Focus (Theory):** Lexical scope, scope chain, closures (reference vs value), var/let/const, temporal dead zone.
- **How to do it:** Before writing code, draw a memory diagram on paper showing where each variable lives. Expose addTask/removeTask/getTasks functions, and

keep everything else private inside the closure. Show it to the AI only for review—struggle on your own first.

- **Connection to next steps:** This createTaskStore module will be transformed into React state in Week 8, so design it well now.

Week 2: Async Engine — Event Loop & Promises

- **What you will build in ASCENT:** A custom MyPromise class + a 'Sync Queue' simulator that will serve as the foundation for Ascent's future API call layer (simulating a few pending 'data sync' operations using setTimeout, including then() chaining and error propagation).
- **Core Focus (Theory):** Call stack, Web API, callback queue vs microtask queue, why a Promise resolves before setTimeout(0), async/await syntactic sugar.
- **How to do it:** First, trace the execution order of a sample async code on paper, then write the code. Your own tracing mistakes are your best teacher here.
- **Connection to next steps:** Understanding this async pattern is the foundation for understanding Express API calls in Week 16 and AI streaming responses in Week 25.

Week 3: OOP, Prototype & Functional Core

- **What you will build in ASCENT:** An Event Emitter class (.on(), .emit(), .off()) for Ascent's internal notification system—this will broadcast events when a task is completed. Along with it, build your own map/filter/reduce toolkit, which will be useful later for filtering tasks/notes.
- **Core Focus (Theory):** Prototype chain, the 4 rules of this binding (implicit/explicit/new/arrow), pure functions, immutability.
- **How to do it:** Write 2 separate code examples for each rule of this. Build a small demo with the Event Emitter where a 'taskCompleted' event is subscribed to/emitted in the console.
- **Connection to next steps:** This EventEmitter concept will transform into WebSocket-based real-time notifications in Week 19.

Week 4: TypeScript Fundamentals — Ascent's Data Model

- **What you will build in ASCENT:** Most important week—Ascent's core types will be defined here: User, Task, Note, Document, StudyGroup. Migrate the modules

from Weeks 1-3 (createTaskStore, Event Emitter) to TypeScript using these types.

- **Core Focus (Theory):** Union/intersection/literal types, generics, utility types (Partial, Pick, Omit, Record), discriminated unions.
- **How to do it:** Before writing types, think: "What is the shape of this data?" Use a discriminated union for Task status: {status: 'todo'} | {status: 'done', completedAt: Date}.
- **Connection to next steps:** This types.ts file will be the backbone of the entire project (frontend + backend) and will be reused repeatedly over the next 23 weeks.

Week 5: TypeScript Tooling — Ascent Repository Skeleton

- **What you will build in ASCENT:** Setup a real project structure: /apps/web (future frontend), /apps/api (future backend), /packages/types (shared types from Week 4) along with ESLint + Prettier + tsconfig.
- **Core Focus (Theory):** interface vs type, mapped/conditional types, tsconfig.json deep dive, declaration files.
- **How to do it:** Write every config file yourself by understanding it, without copy-pasting. This skeleton will be your actual repository where code will be added over the next 22 weeks.
- **Connection to next steps:** UI code will go into /apps/web from Week 7 onwards, and API code will go into /apps/api from Week 16 onwards.

Week 6: CS Fundamentals — Search & Networking Basics

- **What you will build in ASCENT:** A Trie or BST in TypeScript as the foundation for Ascent's 'Search Tasks/Notes' feature to search quickly by title. Additionally, implement a stack/queue (to help understand BullMQ later).
- **Core Focus (Theory):** Linked list/stack/queue/BST, HTTP lifecycle (status codes, headers, CORS), DNS/TCP-IP, browser rendering pipeline.
- **How to do it:** Open Chrome DevTools' Network tab, inspect the request/response headers of a real website, and take notes—this HTTP knowledge will be directly useful during API design in Week 16.
- **Connection to next steps:** Phase 1 is complete. You now have typed logic modules and a project skeleton, but no UI yet. That will be created in Phase 2.

PHASE 2: Frontend Engineering (Week 7-14)

What Ascent's users will see and touch—Dashboard, Task Tracker, Notes, and public portfolio/blog will all be built in this phase.

Week 7: HTML/CSS — Ascent's First Visible Form

- **What you will build in ASCENT:** Three static pages using pure HTML/CSS (no JS): (1) Landing Page, (2) Dashboard layout (sidebar + navbar + task card grid, with placeholder data), (3) 'New Task/Note' creation form.
- **Core Focus (Theory):** Box model, all positioning types, Flexbox & CSS Grid deeply, specificity, cascade, semantic HTML.
- **How to do it:** Pick a design reference (search 'dashboard UI' on Dribbble/Mobbin), recreate it yourself first, and then show it to AI to ask 'why'. Add dark mode (CSS-only) and ARIA labels as a bonus.
- **Connection to next steps:** These three pages will turn into React components starting from Week 8; building a good HTML structure now will save effort later.

Week 8: React Fundamentals — Dashboard Comes to Life

- **What you will build in ASCENT:** Make Week 7's Dashboard completely interactive using React. Full CRUD for Tasks using local state (no backend yet). Tasks will include priority, due date, category, and status.
- **Core Focus (Theory):** Component tree, reconciliation, JSX, props, composition, useState, useEffect (dependency array declaration, not a trick).
- **How to do it:** Convert Week 1's createTaskStore logic into a useState / custom hook. The concept is the same, just the language of React. Observe re-renders using React DevTools.
- **Connection to next steps:** This Task feature is the centerpiece of Ascent—the 'Roadmap Tracker' will remain the primary focus throughout the project.

Week 9: React Advanced Patterns + Typed Component Library

- **What you will build in ASCENT:** A reusable, fully-typed component library for Ascent containing Button, Input, Modal, Dropdown, and a 'Note Editor' component (textarea + markdown preview). Refactor Week 8's Task feature using these components.
- **Core Focus (Theory):** useRef, useCallback, useMemo, custom hooks, useContext, controlled vs uncontrolled, compound components.

- **How to do it:** Build the Dropdown using the compound component pattern. Build and use at least 2 custom hooks (useDebounce, useLocalStorage).
- **Connection to next steps:** The 'Note' feature is introduced to Ascent for the first time here—this will connect with the AI document upload in Week 24.

Week 10: Next.js — Ascent Goes Production-Grade Frontend

- **What you will build in ASCENT:** Port the entire frontend to Next.js App Router. Add dynamic routes: /task/[id], /notes/[id]. Create a placeholder API route (/api/tasks). Deploy it to Vercel.
- **Core Focus (Theory):** Server vs client components, file-based routing, layouts, loading/error states, data fetching & caching.
- **How to do it:** Remember the rule: "Do I need browser interaction (onClick/useState)?" If yes, client component; if no, server component.
- **Connection to next steps:** This /api/tasks route will be replaced by a real Express + PostgreSQL backend in Week 17.

Week 11: State Management — Zustand + TanStack Query

- **What you will build in ASCENT:** Manage UI state (sidebar, theme, modal) with Zustand, and manage 'server state' (currently local placeholder API) with TanStack Query—including optimistic updates so that the UI updates instantly when a task is added.
- **Core Focus (Theory):** Prop drilling issues, caching, invalidation, background refetching, optimistic updates.
- **How to do it:** First, experience the pain of prop drilling yourself (try passing state down a deep tree without Zustand), then solve it. Keep TanStack Query DevTools open.
- **Connection to next steps:** When the real PostgreSQL backend arrives in Week 17, this exact TanStack Query setup will work just by changing the URL—that is the benefit of a good architecture.

Week 12: Tailwind CSS — Ascent's Visual Identity

- **What you will build in ASCENT:** Restyle the entire app using Tailwind. Define your own brand color palette and spacing scale in tailwind.config. Build a real dark mode (controlled and persisted via Zustand state).

- **Core Focus (Theory):** Utility-first philosophy, CSS Modules for scoping, responsive prefixes, custom themes.
- **How to do it:** Design it yourself instead of copying default colors/spacing—this will make Ascent feel like 'your own product', not a template.
- **Connection to next steps:** This design system will persist into Week 14's portfolio and the final product in Weeks 26-27.

Week 13: Testing + Performance

- **What you will build in ASCENT:** Write utility/hook tests with Vitest, test the main flow of the Task feature (add/delete/filter) with React Testing Library, and write an E2E test ('user adds a task → shows up on dashboard') with Playwright. Perform a Lighthouse audit and implement image lazy-loading.
- **Core Focus (Theory):** Core Web Vitals (LCP, INP, CLS), code splitting, testing philosophy (test behavior, not implementation).
- **How to do it:** Before writing tests, think: "What will the user see/do?" Focus on behavior, not implementation details.
- **Connection to next steps:** These tests will run automatically in Week 21's CI/CD pipeline.

Week 14: Frontend Capstone — Ascent Goes Public

- **What you will build in ASCENT:** Fully polish the public-facing part of Ascent. Add a 'Learning Journal' (MDX blog) feature—this is where you will post summaries of your real-life weekly learnings. Fully responsive design, dark mode, deployed, tested, with a proper README.
- **How to do it:** Post your learned concepts in this blog at the end of each week—doing two things at once: practice + portfolio content. Frontend is complete.
- **Connection to next steps:** In Phase 3, a real backend will be placed behind this UI.

PHASE 3: Backend Engineering (Week 15-22)

A real server, database, authentication, and real-time system will be placed behind the frontend—Ascent now becomes a true production app.

Week 15: Node.js Runtime — Ascent's Server Foundation

- **What you will build in ASCENT:** A raw HTTP server (without Express, using only the http module) that serves a simple /health and an in-memory /tasks endpoint for Ascent.
- **Core Focus (Theory):** libuv, thread pool, non-blocking I/O, Node event loop vs browser event loop, streams & buffers.
- **How to do it:** Do not skip this week before moving to Express—this will reveal exactly what 'magic' Express does for you.
- **Connection to next steps:** This raw server will be replaced with Express in Week 16, but the underlying concepts will stick.

Week 16: Express + REST API — Ascent's Real Backend Begins

- **What you will build in ASCENT:** A complete REST API for Ascent—for Users, Tasks, and Notes resources, including TypeScript + Zod validation. This code will live in the /apps/api folder created in Week 5.
- **Core Focus (Theory):** Middleware pipeline, error-handling middleware, REST resource naming, HTTP methods, status codes, versioning.
- **How to do it:** Create a Postman/Thunder Client collection for all endpoints. Reuse the shared types (@ascent/types) from Week 4 here.
- **Connection to next steps:** Week 10's placeholder /api/tasks route will now be replaced with this real Express API—frontend code will change minimally because TanStack Query was already set up.

Week 17: PostgreSQL + Prisma — Ascent Data Becomes Persistent

- **What you will build in ASCENT:** Persist Tasks, Notes, and Users into Postgres using Prisma. Write a few raw SQL queries (joins, aggregations) first, then build the Prisma schema. Connect the frontend to the real backend now.
- **Core Focus (Theory):** Normalization (1NF-3NF), primary/foreign keys, indexing, ACID transactions, EXPLAIN ANALYZE.
- **How to do it:** For the first time, refreshing the browser will keep the Task data intact (stored in DB instead of in-memory)—this moment is the first signal of becoming a 'real app'. Compare Week 6's Trie/BST search logic with Postgres search queries to understand the difference.

Week 18: Authentication + Security — Ascent Becomes Private

- **What you will build in ASCENT:** A complete auth system: registration, login, bcrypt hashing, JWT access + refresh tokens, and protected routes. Now, every user will have their own private Tasks/Notes.
- **Core Focus (Theory):** bcrypt cost factor, JWT statelessness, preventing SQL injection/XSS/CSRF, CORS configuration, security headers.
- **How to do it:** Try to intentionally run a SQL injection on your own API (locally), then fix it. Build the login/register pages using the component library from Week 9.
- **Connection to next steps:** Week 14's Learning Journal blog can now have a per-user private/public toggle.

Week 19: Redis, Queues, Storage, WebSocket

- **What you will build in ASCENT:** Four features on a small scale: (1) Redis caching for dashboard stats, (2) BullMQ for a 'Daily Reminder' email job, (3) S3-compatible storage ($R2$/MinIO) for uploading study documents, (4) Socket.io for real-time 'Study Group presence'.
- **Core Focus (Theory):** Cache-aside pattern & TTL, why job queues are needed (heavy work outside the request cycle), WebSocket basics.
- **How to do it:** Do not do everything at once; add them one by one and test separately.
- **Connection to next steps:** This week's document upload feature will be the raw material for Week 24's AI RAG system—the most critical foundation for Phase 4.

Week 20: System Design — What if Ascent Scales?

- **What you will build in ASCENT:** No coding—Design Document: 'If Ascent gets 100k users, what will break and how will we fix it?' Write a 1-2 page answer (diagram + trade-offs) covering rate limiting, DB replication, and caching strategy.
- **Core Focus (Theory):** CAP theorem, load balancing, replication/sharding, microservices vs monolith, message queues.
- **How to do it:** Draw an architecture diagram using Excalidraw or on paper. Take notes after reading Chapters 1-2 of Kleppmann's book, and connect them with Ascent.
- **Connection to next steps:** This design thinking will be useful for Week 21's deployment and Week 26-27's scaling decisions.

Week 21: DevOps — Ascent Becomes Deployable

- **What you will build in ASCENT:** Dockerize the entire backend (API + Postgres + Redis) using docker-compose. Create a GitHub Actions CI/CD pipeline (Week 13's tests will run automatically; if they pass, deploy). Deploy to Railway/Render.
- **Core Focus (Theory):** Docker images/containers/volumes/networks, multi-stage builds, CI/CD concepts.
- **How to do it:** Write the Dockerfile yourself with understanding, and try a multi-stage build to reduce image size.
- **Connection to next steps:** Now Ascent is fully live with frontend (Vercel) + backend (Railway/Render) working together in production.

Week 22: Backend Capstone — Study Groups (Real-time Collaboration)

- **What you will build in ASCENT:** 'Study Group' feature where multiple users can edit a shared note/roadmap document simultaneously in real-time (via WebSockets), combining Redis caching, BullMQ notifications, and pino logging all together.
- **Core Focus (Theory):** Real-time sync patterns, proper error handling, Zod validation, structured logging.
- **How to do it:** Write in an Architecture Decision Record (ADR) style in the README explaining why you chose Redis and why BullMQ.
- **Connection to next steps:** Backend Phase is complete. This documentation will set you apart in interviews. Now, in Phase 4, Ascent will get smart.

PHASE 4: AI Integration (Week 23-27)

Ascent learns to read your documents, answer questions, and directly perform tasks—this is what will make your profile stand out.

Week 23: AI/ML Fundamentals — First AI Feature

- **What you will build in ASCENT:** An 'Ask AI' button inside a Task that calls the OpenAI Chat Completions API directly to explain a task-related concept (using system/user/assistant roles). No streaming or RAG yet—just a basic API call.
- **Core Focus (Theory):** Tokens, embeddings concept (cosine similarity), high-level idea of transformer/attention, temperature, context window.

- **How to do it:** Run the same prompt with temperatures 0, 0.5, 1, and 1.5 to observe output differences. Tokenize your own text using the OpenAI tokenizer tool.
- **Connection to next steps:** This basic 'Ask AI' button will be transformed into a streaming chat UI in Week 25.

Week 24: RAG System — Ascent's AI Mentor is Born

- **What you will build in ASCENT:** An 'AI Mentor' chat using documents (PDFs/notes) uploaded in Week 19—chunking → embedding → storing in pgvector → similarity search → answering with citations. Users can upload study materials and ask questions.
- **Core Focus (Theory):** Every stage of the RAG pipeline, chunking strategy, vector similarity search, pgvector.
- **How to do it:** Test each stage of the pipeline separately (print and check if chunking is correct, then embedding, then search). Experiment with chunk sizes (300 vs 800 tokens).
- **Connection to next steps:** This is Ascent's most critical AI feature—the main foundation for the flagship project in Weeks 26-27.

Week 25: Streaming + Tool Use — AI Mentor Performs Actual Work

- **What you will build in ASCENT:** Upgrade Week 23's basic 'Ask AI' into a streaming chat UI using the Vercel AI SDK. Add a 'Smart Summarizer' for long notes. Empower the AI Mentor with function calling to directly create Tasks—the user can say 'Add a task to revise closures tomorrow', and the AI will invoke the createTask function.
- **Core Focus (Theory):** Streaming UI via SSE/WebSocket, prompt engineering (system prompt, few-shot, chain-of-thought), tool/function calling.
- **How to do it:** Build the UI using Vercel AI SDK's useChat hook, but also try doing it once with raw SSE to understand the underlying mechanism.
- **Connection to next steps:** AI features are now working together. In Weeks 26-27, these will be polished to build the final product.

Week 26-27: Final Capstone — Ascent is Complete

- **What you will build in ASCENT:** Polish everything together: auth, subscription/usage limits (document upload limits on the free tier), admin

dashboard (users and usage statistics), RAG-driven AI Mentor (with citation + streaming + tool calling), Study Groups, and fully deployed.

- **Core Focus:** End-to-end polish, production readiness, documentation.
- **How to do it:** Prevent scope creep—polish what you have already built from Weeks 8-25, avoid the temptation of adding new features. Build a comprehensive architecture diagram in the README. This will be the biggest showpiece on your resume.

Final Words — After 27 Weeks

Four rules to remember every week:

1. **Push to GitHub daily:** Ascent is a single repository (or monorepo), so the commit history itself will tell the story of your growth.
2. **Struggle first, AI review later:** Maintain the method that worked in Week 7 (recreate yourself first → ask AI 'why') throughout the entire 27 weeks.
3. **Write Ascent's own blog at the end of every week:** Starting from Week 14, this habit becomes a product feature; you won't need to allocate separate time for it.
4. **Do not expand the scope:** Even if tempted by new features, stick to what the roadmap dictates. Depth is important, not breadth.

Is AI making us obsolete? The answer to this anxiety lies within this roadmap. An engineer who only "writes code using AI" is the one at risk. An engineer who understands *why* a closure is needed, *why* RAG works this way, and *why* Redis caching is necessary—to them, AI is a tool, not a competitor. The entire process of building Ascent is an exercise in understanding that "why".

Before starting Week 1:

Create a repository, name it (Ascent or your preferred name), and write a README where you explain in one paragraph—what this project is, why you are building it, and what it will become after 27 weeks. Then, open Week 1.